

操作系统实验报告

姓名：李宗杭 学号：2311451 学院：软件学院

实验日期：2025.11.5 星期三 11-12 节

实验题目：用户态进程列出程序

一、实验内容

编写一个 c/c++ 程序，在用户态列出所有进程信息，模拟 `ps -ef` 的结果。

二、实现步骤

1、环境准备

(1) 安装/更新核心编译工具

```
sudo apt-get install -y build-essential
```

(2) 选择 IDE

```
VSCode + Remote-SSH + gcc
```

2、认识/proc 目录

/proc 目录是一个虚拟文件系统，内核会实时将所有进程的完整信息存储在此目录下。每个进程对应一个以其 PID 命名的子目录，子目录中包含多个文件，存储着进程的 UID、PID、PPID、命令行等关键信息。

文件	作用	关键内容
stat	进程状态信息(紧凑)	PID(进程ID)、PPID(父进程ID)、C(CPU利用率)、STIME(启动时间)、TTY(终端设备号)、TIME(占用CPU时间)
status	进程状态信息(易读)	UID(用户ID)、GID(组ID)
cmdline	进程启动时完整命令行	CMD(命令及参数)

3、代码编写

(0) 头文件引入与宏定义

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <dirent.h>
4  #include <string.h>
5  #include <unistd.h>
6  #include <sys/stat.h>
7  #include <pwd.h>
8  #include <time.h>
9  #include <errno.h>
10 #include <ctype.h>
11
12 // 定义缓冲区大小
13 #define MAX_BUF 4096
14 #define MAX_CMD 1024
```

(1) PID 目录判断

```
16 // 1. 判断目录名是否为纯数字 (PID目录)
17 int is_pid_dir(const char *name) {
18     for (int i = 0; name[i] != '\0'; i++) {
19         if (!isdigit(name[i])) {
20             return 0;
21         }
22     }
23     return 1;
24 }
```

(2) 从 status 解析 UID

```
26 // 2. 从status解析UID
27 uid_t get_uid_from_status(const char *pid_path) {
28     char status_path[MAX_BUF];
29     snprintf(status_path, sizeof(status_path), "%s/status", pid_path);
30
31     FILE *fp = fopen(status_path, "r");
32     if (fp == NULL) {
33         return -1;
34     }
35
36     char buf[MAX_BUF];
37     uid_t uid = -1;
38     // 查找以"Uid:"开头的行
39     while (fgets(buf, sizeof(buf), fp) != NULL) {
40         if (strncmp(buf, "Uid:", 4) == 0) {
41             sscanf(buf, "Uid: %d", &uid); // 解析第一个UID (Real UID)
42             break;
43         }
44     }
45
46     fclose(fp);
47     return uid;
48 }
```

(3) 从 status 解析 PPID

```
50 // 3. 从status解析PPID
51 pid_t get_ppid_from_status(const char *pid_path) {
52     char status_path[MAX_BUF];
53     snprintf(status_path, sizeof(status_path), "%s/status", pid_path);
54
55     FILE *fp = fopen(status_path, "r");
56     if (fp == NULL) {
57         return -1;
58     }
59
60     char buf[MAX_BUF];
61     pid_t ppid = -1;
62     // 查找以"PPid:"开头的行
63     while (fgets(buf, sizeof(buf), fp) != NULL) {
64         if (strncmp(buf, "PPid:", 5) == 0) {
65             sscanf(buf, "PPid: %d", &ppid);
66             break;
67         }
68     }
69
70     fclose(fp);
71     return ppid;
72 }
```

(4) 从 cmdline 解析命令行

```
74 // 4. 从cmdline解析命令行（空字符分隔参数）
75 void get_cmd_from_cmdline(const char *pid_path, char *cmd) {
76     char cmdline_path[MAX_BUF];
77     snprintf(cmdline_path, sizeof(cmdline_path), "%s/cmdline", pid_path);
78
79     FILE *fp = fopen(cmdline_path, "r");
80     if (fp == NULL) {
81         cmd[0] = '\0';
82         return;
83     }
84
85     char buf[MAX_CMD];
86     int len = fread(buf, 1, sizeof(buf)-1, fp); // 读取文件内容
87     fclose(fp);
88
89     // 将cmdline中的空字符\0替换为空格
90     cmd[0] = '\0';
91     for (int i = 0; i < len; i++) {
92         if (buf[i] == '\0') {
93             strcat(cmd, " ");
94         } else {
95             strncat(cmd, &buf[i], 1);
96         }
97     }
98
99     // 去除末尾多余的空格
100     len = strlen(cmd);
101     if (len > 0 && cmd[len-1] == ' ') {
102         cmd[len-1] = '\0';
103     }
104 }
```

(5) 从 stat 解析 utime、stime、starttime、tty_nr

[illegible]

(6) 将 tty nr 转换为终端名

```

127 // 6. 将tty_nr转换为终端名
128 char *tty_nr_to_name(int tty_nr) {
129     char *name = malloc(32);
130     if (tty_nr == 0) {
131         strcpy(name, "?"); // 无终端
132     } else if (tty_nr >= 4 && tty_nr <= 63) {
133         // 虚拟终端 (tty1-tty63): tty_nr = 4 + (n-1) → n = tty_nr - 3
134         sprintf(name, "tty%d", tty_nr - 3);
135     } else if (tty_nr >= 128) {
136         // 伪终端 (pts/0-pts/n): tty_nr = 128 + n → n = tty_nr - 128
137         sprintf(name, "pts/%d", tty_nr - 128);
138     } else {
139         strcpy(name, "?");
140     }
141     return name;
142 }

```

(7) 将 jiffies 转换为 HH:MM:SS

```
144 // 7. 将jiffies转换为HH:MM:SS
145 char *format_time(unsigned long jiffies) {
146     char *time_str = malloc(16);
147     unsigned long total_sec = jiffies / sysconf(_SC_CLK_TCK);
148     unsigned int h = total_sec / 3600;
149     unsigned int m = (total_sec % 3600) / 60;
150     unsigned int s = total_sec % 60;
151     sprintf(time_str, "%02d:%02d:%02d", h, m, s);
152     return time_str;
153 }
```

(8) 将 starttime 转换为启动时间 HH:MM

```
155 // 8. 将starttime转换为启动时间HH:MM
156 char *format_starttime(unsigned long starttime) {
157     char *stime_str = malloc(16);
158     unsigned long start_sec = starttime / sysconf(_SC_CLK_TCK);
159
160     // 读取/proc/uptime获取系统运行时间
161     double uptime_seconds = 0.0;
162     FILE *uptime_fp = fopen("/proc/uptime", "r");
163     if (uptime_fp != NULL) {
164         fscanf(uptime_fp, "%lf", &uptime_seconds);
165         fclose(uptime_fp);
166     }
167
168     // 计算进程启动的绝对时间
169     time_t now = time(NULL);
170     time_t boot_time = now - (time_t)(uptime_seconds - start_sec);
171     struct tm *boot_tm = localtime(&boot_time);
172
173     if (boot_tm != NULL) {
174         sprintf(stime_str, "%02d:%02d", boot_tm->tm_hour, boot_tm->tm_min);
175     } else {
176         strcpy(stime_str, "00:00");
177     }
178
179     return stime_str;
180 }
```

(9) 计算 CPU 使用率

```
182 // 9. 简化计算CPU使用率（相对占比，仅作演示）
183 float calculate_cpu_usage(unsigned long utime) {
184     // 实际ps的C字段是进程占用CPU的百分比（基于系统总CPU时间）
185     // 此处简化为: utime / 总CPU时间（取前100个进程的平均）
186     static unsigned long total_utime = 0;
187     static int count = 0;
188     total_utime += utime;
189     count++;
190     if (count == 0 || total_utime == 0) {
191         return 0.0;
192     }
193     return (float)utime / (total_utime / 100) > 100 ? 100.0 : (float)utime / (total_utime / 100);
194 }
```

(10) 主函数

```
196 int main() {
197     DIR *proc_dir;
198     struct dirent *dir_entry;
199     char pid_path[MAX_BUF]; // 进程目录路径
200     char cmd[MAX_CMD];      // 进程命令行
201     uid_t uid;              // 进程UID
202     pid_t ppid;             // 父进程ID
203     unsigned long utime, stime, starttime; // CPU时间和启动时间戳
204     int tty_nr;             // 终端编号
205     float cpu_usage;        // CPU使用率
206     struct passwd *pw;      // 用于将UID转换为用户名
207
208     // 打开/proc目录
209     proc_dir = opendir("/proc");
210     if (proc_dir == NULL) {
211         perror("opendir /proc failed");
212         exit(EXIT_FAILURE);
213     }
214     // 输出表头
215     printf("%-8s %-6s %-6s %-4s %-8s %-6s %-10s %s\n",
216           "UID", "PID", "PPID", "C", "STIME", "TTY", "TIME", "CMD");
217     // 遍历/proc目录
218     while ((dir_entry = readdir(proc_dir)) != NULL) {
219         // 只处理数字命名的目录
220         if (!is_pid_dir(dir_entry->d_name)) {
221             continue;
222         }
223         // 构建进程目录路径
224         snprintf(pid_path, sizeof(pid_path) - 6, "/proc/%s", dir_entry->d_name);
225         // 解析各个字段
226         pid_t pid = atoi(dir_entry->d_name);
227         // 解析UID
228         uid = get_uid_from_status(pid_path);
229         pw = getpwuid(uid); // 将UID转换为用户名
230         char *username = (pw != NULL) ? pw->pw_name : "unknown";
231         // 解析PPID
232         ppid = get_ppid_from_status(pid_path);
233         // 解析stat文件中的CPU时间、启动时间、终端编号
234         get_stat_info(pid_path, &utime, &stime, &starttime, &tty_nr);
235         // 计算CPU使用率
236         cpu_usage = calculate_cpu_usage(utime);
237         // 格式化启动时间 (STIME)
238         char *stime_str = format_starttime(starttime);
239         // 转换终端名 (TTY)
240         char *tty_name = tty_nr_to_name(tty_nr);
241         // 格式化累计CPU时间 (TIME)
242         unsigned long total_jiffies = utime + stime;
243         char *time_str = format_time(total_jiffies);
244         // 解析命令行 (CMD)
245         get_cmd_from_cmdline(pid_path, cmd);
246         // 若cmd为空, 从stat文件读取进程名
247         if (strlen(cmd) == 0) {
248             char stat_path[MAX_BUF];
249             snprintf(stat_path, sizeof(stat_path), "%s/stat", pid_path);
250             FILE *stat_fp = fopen(stat_path, "r");
251             if (stat_fp != NULL) {
252                 fscanf(stat_fp, "%*d (%[^)])", cmd); // 匹配"(进程名)"格式
253                 fclose(stat_fp);
254             } else {
255                 strcpy(cmd, "[unknown]");
256             }
257         }
258         // 格式化输出
259         printf("%-10s %-6d %-6d %-4.1f %-8s %-6s %-10s %s\n",
260               username, pid, ppid, cpu_usage, stime_str, tty_name, time_str, cmd);
261
262         free(stime_str);
263         free(time_str);
264         free(tty_name);
265     }
266
267     // 6. 关闭目录句柄
268     closedir(proc_dir);
269     return 0;
270 }
```

4、编译与运行

```
gcc ps_emulator.c -o ps_emulator -Wall
```

```
./ps_emulator
```

```
LZH2311451@LZH2311451-pc:~$ gcc ps_emulator.c -o ps_emulator -Wall
ps_emulator.c: In function 'main':
ps_emulator.c:249:55: warning: '/stat' directive output may be truncated writing 5 bytes into a region of size between 1 and 4096 [-Wformat-truncation=]
    249 |         snprintf(stat_path, sizeof(stat_path), "%s/stat", pid_path);
        |         ~~~~~^~~~~
ps_emulator.c:249:13: note: 'snprintf' output between 6 and 4101 bytes into a destination of size 4096
    249 |         snprintf(stat_path, sizeof(stat_path), "%s/stat", pid_path);
        |         ~~~~~^~~~~
LZH2311451@LZH2311451-pc:~$ ./ps_emulator
```

编译时会出现一个警告：snprintf 可能导致输出截断。

这是因为 pid_path 的大小是 MAX_BUF (4096)，拼接"/stat"后，总长度可能达到 4096 + 5 = 4101，而 stat_path 的大小同样是 4096。编译器进行静态分析时，认为存在超出缓冲区大小的风险，因此抛出截断警告。但尽管实际运行中 pid_path 是 /proc/[PID]，长度远小于 4096，不会真的截断，所以可以忽略此警告。

5、分析结果

首先可以看到程序能正确运行，且输出结果符合预期

```
LZH2311451@LZH2311451-pc:~$ ./ps_emulator
```

UID	PID	PPID	C	STIME	TTY	TIME	CMD
root	1	0	0.0	12:02	?	00:00:15	/sbin/init splash
root	2	0	0.0	12:02	?	00:00:00	kthreadd
root	3	2	0.0	12:02	?	00:00:00	pool_workqueue_release
root	4	2	0.0	12:02	?	00:00:00	kworker/R-rcu_gp
root	5	2	0.0	12:02	?	00:00:00	kworker/R-sync_wq
root	6	2	0.0	12:02	?	00:00:00	kworker/R-kvfree_rcu_reclaim
root	7	2	0.0	12:02	?	00:00:00	kworker/R-slub_flushwq
root	8	2	0.0	12:02	?	00:00:00	kworker/R-netns
root	12	2	0.0	12:02	?	00:00:00	kworker/u512:0-ipv6_addrconf
root	13	2	0.0	12:02	?	00:00:00	kworker/R-mm_percpu_wq
root	14	2	2.5	12:02	?	00:00:01	ksoftirqd/0
root	15	2	0.0	12:02	?	00:00:31	rcu_preempt
root	16	2	0.0	12:02	?	00:00:00	rcu_exp_par_gp_kthread worker/1
root	17	2	0.0	12:02	?	00:00:00	rcu_exp_gp_kthread_worker
root	18	2	0.0	12:02	?	00:00:00	migration/0
root	19	2	0.0	12:02	?	00:00:00	idle_inject/0
root	20	2	0.0	12:02	?	00:00:00	cpuhp/0
root	21	2	0.0	12:02	?	00:00:00	cpuhp/1
root	22	2	0.0	12:02	?	00:00:00	idle_inject/1
root	23	2	0.0	12:02	?	00:00:03	migration/1
root	24	2	0.0	12:02	?	00:00:00	ksoftirqd/1
root	27	2	0.0	12:02	?	00:00:00	cpuhp/2
root	28	2	0.0	12:02	?	00:00:00	idle_inject/2
root	29	2	0.0	12:02	?	00:00:03	migration/2
root	30	2	2.0	12:02	?	00:00:00	ksoftirqd/2
root	33	2	0.0	12:02	?	00:00:00	cpuhp/3
root	34	2	0.0	12:02	?	00:00:00	idle_inject/3
root	35	2	0.0	12:02	?	00:00:03	migration/3
root	36	2	0.0	12:02	?	00:00:00	ksoftirqd/3
root	39	2	0.0	12:02	?	00:00:00	cpuhp/4
root	40	2	0.0	12:02	?	00:00:00	idle_inject/4
root	41	2	0.0	12:02	?	00:00:03	migration/4
root	42	2	0.0	12:02	?	00:00:00	ksoftirqd/4
root	45	2	0.0	12:02	?	00:00:00	cpuhp/5

但在继续查看后续输出时，却发现部分内容与系统 ps -ef 的输出存在明显差别。

这是因为为了保证输出格式整齐、可读性强，ps -ef 会对过长的命令行进行字符长度截断，超出一定长度的部分会被省略，如图中所示 61781 进程命令行信息被截断为短格式，就是这个原因。此外，ps 还会优先解析进程的“核心执行命令”，忽略部分冗长的命令行参数（如 VS Code 的 --connection-token、--socket-path 等），聚焦于用户可识别的核心命令。


```

LZH2311451 4907 1 1.6 12:04 ? 00:00:51 /opt/qaxsafe/sqaxsafe4forcnos 5 --minishow
LZH2311451 7099 1601 3.1 12:05 ? 00:01:02 /usr/bin/kylin-software-center-plugin-synchrodata
root 7206 1 2.9 12:05 ? 00:01:03 /usr/bin/python3 /usr/bin/kylin-assistant-systemdaemon.py
LZH2311451 10148 1601 0.1 12:09 ? 00:00:03 /usr/bin/kylin-background-upgrade --check-upgrade
root 41942 2 0.0 16:22 ? 00:00:00 kworker/7:2H-kblockd
root 45554 2 0.0 17:14 ? 00:00:04 kworker/u513:3-events_unbound
root 48164 2 0.0 17:23 ? 00:00:00 kworker/1:2H
LZH2311451 52141 1 6.4 18:00 ? 00:02:14 /usr/bin/peony --show-properties
root 57699 2 0.0 18:36 ? 00:00:00 kworker/2:1H-kblockd
root 57729 2 0.0 18:36 ? 00:00:02 kworker/u514:1-events_unbound
root 61557 2 0.0 19:25 ? 00:00:01 kworker/u514:2-kvfree_rcu_reclaim
LZH2311451 61781 1 0.0 19:26 ? 00:00:00 sh /home/LZH2311451/.vscode-server/cli/servers/Stable-618725e67565b290ba4da6fe2d29f8fa1d4e3622/server/bin/code-server --connection-token=remotessh --accept-server-license-terms --start-server --enable-remote-auto-shutdown --socket-path=/tmp/code-d176ea6-ed7e-49de-b870-2f43ace72713
LZH2311451 61785 61781 3.3 19:26 ? 00:01:08 /home/LZH2311451/.vscode-server/cli/servers/Stable-618725e67565b290ba4da6fe2d29f8fa1d4e3622/server/node /home/LZH2311451/.vscode-server/cli/servers/Stable-618725e67565b290ba4da6fe2d29f8fa1d4e3622/server/out/server-main.js --connection-token=remotessh --accept-server-license-terms --start-server --enable-remote-auto-shutdown --socket-path=/tmp/code-d176ea6-ed7e-49de-b870-2f43ace72713
LZH2311451 61823 61785 0.2 19:26 ? 00:00:02 /home/LZH2311451/.vscode-server/cli/servers/Stable-618725e67565b290ba4da6fe2d29f8fa1d4e3622/server/node /home/LZH2311451/.vscode-server/cli/servers/Stable-618725e67565b290ba4da6fe2d29f8fa1d4e3622/server/out/bootsrap-fork --type=ptyHost --logsPath /home/LZH2311451/.vscode-server/data/logs/20251228T192627
root 61874 2 0.0 19:26 ? 00:00:00 kworker/5:2H-kblockd
root 61875 2 0.0 19:26 ? 00:00:00 kworker/2:2H-kblockd
root 61944 1020 0.0 19:26 ? 00:00:00 sshd: LZH2311451 [priv]
LZH2311451 61963 61944 0.2 19:27 ? 00:00:30 sshd: LZH2311451@notty
LZH2311451 61964 61963 0.0 19:27 ? 00:00:00 sh
LZH2311451 61982 61964 0.7 19:27 ? 00:00:29 /home/LZH2311451/.vscode-server/code-618725e67565b290ba4da6fe2d29f8fa1d4e3622 command-shell --cli-data-dir /home/LZH2311451/.vscode-server/cli --parent-process-id 61964 --on-host=127.0.0.1 --on-port
LZH2311451 62030 61785 18.6 19:27 ? 00:06:13 /home/LZH2311451/.vscode-server/cli/servers/Stable-618725e67565b290ba4da6fe2d29f8fa1d4e3622/server/node --dns-result-order=ipv4first /home/LZH2311451/.vscode-server/cli/servers/Stable-618725e67565b290ba4da6fe2d29f8fa1d4e3622/server/out/bootsrap-fork --type=extensionHost --transformURIs --useHostProxy=false
LZH2311451 62041 61785 3.2 19:27 ? 00:01:14 /home/LZH2311451/.vscode-server/cli/servers/Stable-618725e67565b290ba4da6fe2d29f8fa1d4e3622/server/node /home/LZH2311451/.vscode-server/cli/servers/Stable-618725e67565b290ba4da6fe2d29f8fa1d4e3622/server/out/bootsrap-fork --type=fileWatcher
root 65835 2 0.0 19:46 ? 00:00:00 kworker/4:0H
root 67200 2 0.0 19:57 ? 00:00:00 kworker/6:2H
root 68754 2 0.0 20:18 ? 00:00:00 kworker/4:2H-kblockd
root 68950 2 0.0 20:21 ? 00:00:00 kworker/u516:0-ttm

```

root	7206	1	0	12:05 ?	00:01:03	/usr/bin/python3 /usr/bin/kylin-assistant-systemdaemon.py
LZH2311+	10148	1601	0	12:09 ?	00:00:03	/usr/bin/kylin-background-upgrade --check-upgrade
root	41942	2	0	16:22 ?	00:00:00	[kworker/7:2H-kblockd]
root	45554	2	0	17:14 ?	00:00:04	[kworker/u513:3-events_unbound]
root	48164	2	0	17:23 ?	00:00:00	[kworker/1:2H]
LZH2311+	52141	1	0	18:00 ?	00:02:15	/usr/bin/peony --show-properties
root	57699	2	0	18:36 ?	00:00:00	[kworker/2:1H-kblockd]
root	57729	2	0	18:36 ?	00:00:02	[kworker/u514:1-events_unbound]
root	61557	2	0	19:25 ?	00:00:01	[kworker/u514:2-events_unbound]
LZH2311+	61781	1	0	19:26 ?	00:00:00	sh /home/LZH2311451/.vscode-server/cli/servers/Stable-618725e67565b290ba4d
LZH2311+	61785	61781	0	19:26 ?	00:01:08	/home/LZH2311451/.vscode-server/cli/servers/Stable-618725e67565b290ba4da6f
LZH2311+	61823	61785	0	19:26 ?	00:00:02	/home/LZH2311451/.vscode-server/cli/servers/Stable-618725e67565b290ba4da6f
root	61874	2	0	19:26 ?	00:00:00	[kworker/5:2H-kblockd]
root	61875	2	0	19:26 ?	00:00:00	[kworker/2:2H-kblockd]
root	61944	1020	0	19:26 ?	00:00:00	sshd: LZH2311451 [priv]
LZH2311+	61963	61944	0	19:27 ?	00:00:30	sshd: LZH2311451@notty
LZH2311+	61964	61963	0	19:27 ?	00:00:00	sh
LZH2311+	61982	61964	0	19:27 ?	00:00:30	/home/LZH2311451/.vscode-server/code-618725e67565b290ba4da6fe2d29f8fald4e3
LZH2311+	62030	61785	3	19:27 ?	00:06:14	/home/LZH2311451/.vscode-server/cli/servers/Stable-618725e67565b290ba4da6f
LZH2311+	62041	61785	0	19:27 ?	00:01:15	/home/LZH2311451/.vscode-server/cli/servers/Stable-618725e67565b290ba4da6f
root	65835	2	0	19:46 ?	00:00:00	[kworker/4:0H]
root	67209	2	0	19:57 ?	00:00:00	[kworker/6:2H]
root	68754	2	0	20:18 ?	00:00:00	[kworker/4:2H-kblockd]
root	68950	2	0	20:21 ?	00:00:00	[kworker/u516:0-ttm]
root	69218	2	0	20:24 ?	00:00:00	[kworker/1:0H-kblockd]
root	69389	2	0	20:26 ?	00:00:00	[kworker/u513:1-kvfree_rcu_reclaim]
root	69458	2	0	20:27 ?	00:00:00	[kworker/3:1H-kblockd]
root	72148	2	0	20:33 ?	00:00:01	[kworker/u513:0-kvfree_rcu_reclaim]
root	72187	2	0	20:33 ?	00:00:00	[kworker/6:0H-kblockd]
root	72485	2	0	20:38 ?	00:00:00	[kworker/3:0H]
root	73589	2	0	20:54 ?	00:00:00	[kworker/u516:2-ttm]
root	74633	2	0	21:01 ?	00:00:00	[kworker/7:0H-kblockd]
root	74689	2	0	21:01 ?	00:00:00	[kworker/0:0H-kblockd]
root	76086	2	0	21:20 ?	00:00:00	[kworker/u517:1-ttm]
root	76594	2	0	21:26 ?	00:00:01	[kworker/5:1-events]
root	80350	2	0	21:47 ?	00:00:00	[kworker/u517:3]
LZH2311+	80475	1	0	21:48 ?	00:00:22	mate-terminal
LZH2311+	80488	80475	0	21:48 pts/3	00:00:00	bash
root	80841	2	0	21:53 ?	00:00:01	[kworker/1:1-events]

三、结论

本次实验通过 C 语言编程成功实现了用户态进程信息列出功能，模拟了 `ps -ef` 命令的输出效果。程序利用 Linux 系统/`proc` 虚拟文件系统的特性，遍历其中以 PID 命名的进程目录，读取 `stat`、`status`、`cmdline` 等关键文件，提取并转换 UID、PID、PPID、CPU 时间、启动时间、终端、命令行等字段，经格式化处理后输出与 `ps -ef` 一致的进程信息。加深了我对 Linux 系统进程信息的存储机制与用户态访问方式的理解。